输入输出系统
（设备管理）

设备管理概述及子系统

磁盘管理与组织

考研要求

- **I/O**管理概述
  - **I/O**设备
  - **I/O**管理目标
  - **I/O**管理功能
  - **I/O**应用接口
  - **I/O**控制方式
- **I/O**核心子系统
  - **I/O**调度概念
  - 高速缓存与缓冲区
  - 设备分配与回收
  - 假脱机技术（**SPOOLing**）
  - 出错处理
- 磁盘的组织与管理
  - 磁盘的结构
  - 磁盘调度算法
  - 磁盘的管理

一 I/O系统功能

I/o系统主要目标：高效、通用

1. 隐藏物理设备的细节

2. 与设备的无关性

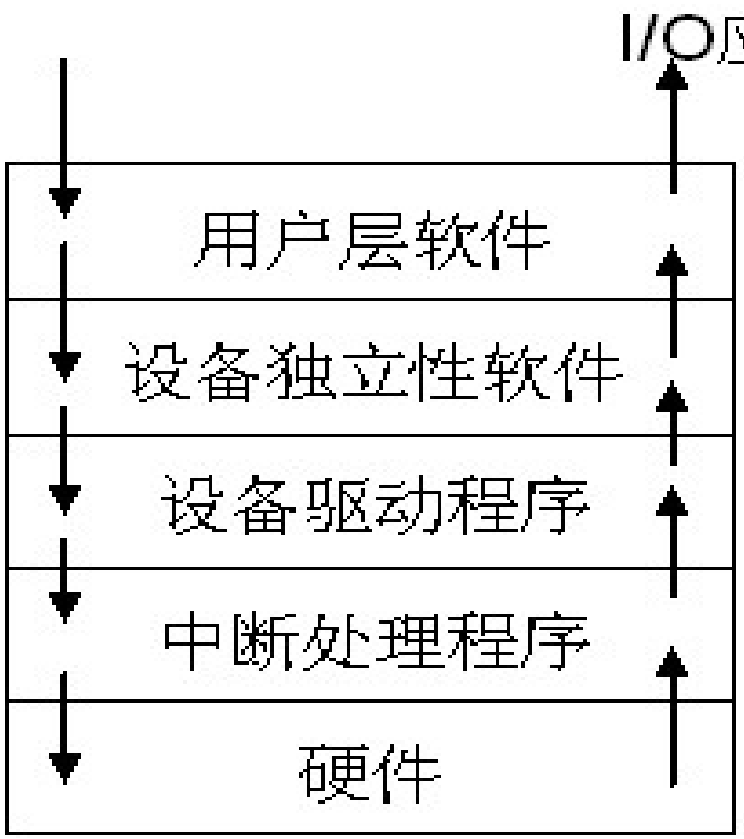
3. 提高处理机和I/O设备的利用率

4. 对I/O设备进行控制

5. 能确保对设备的正确共享

6. 错误处理

I/O软件的层次结构



产生I/O请求、格式化I/O、Spooling

映射、保护、分块、缓冲、分配

设置设备寄存器；检查状态

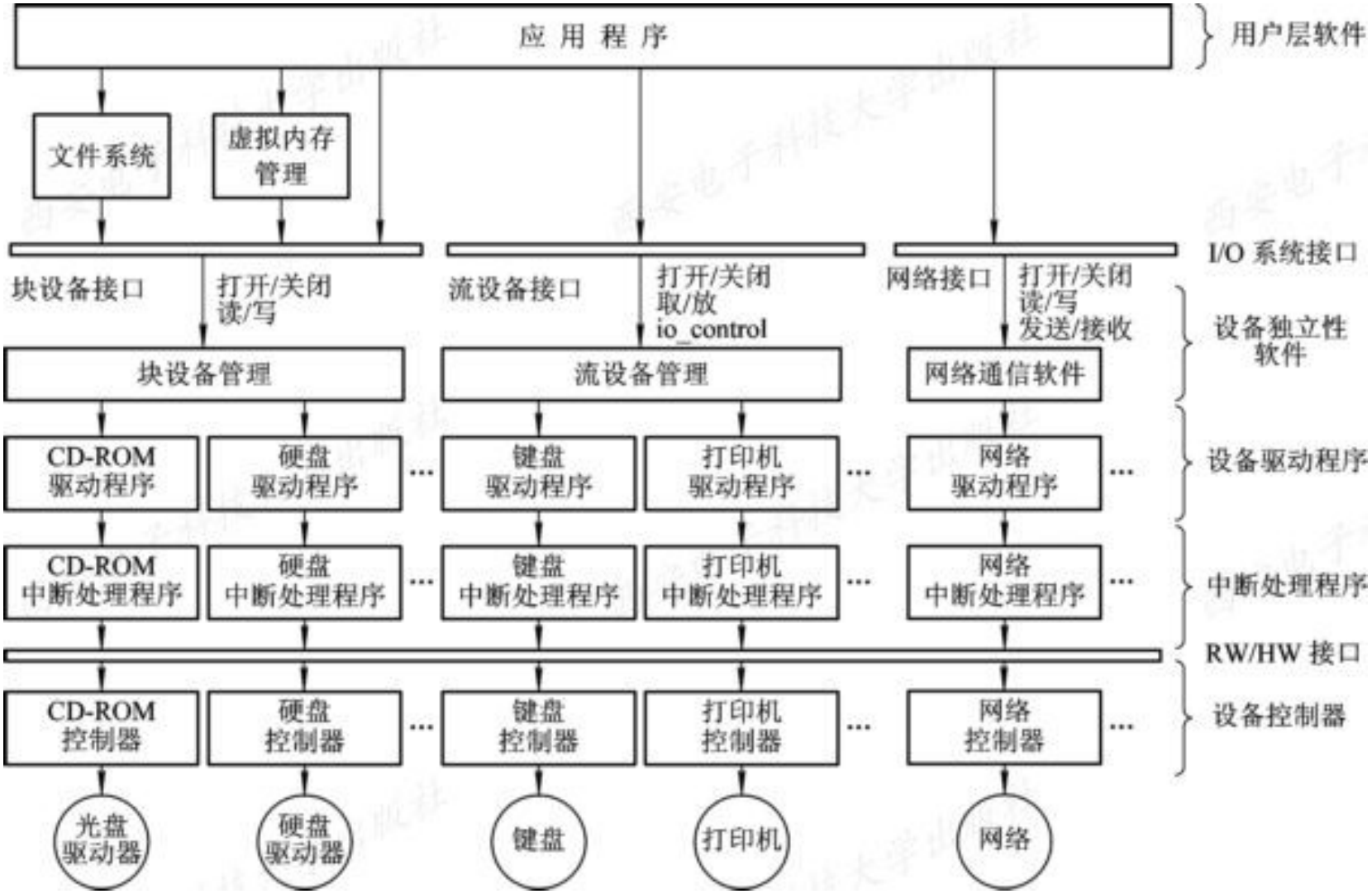
用于保存被中断进程的**CPU**环境，转入相应的中断处理程序进行处理，处理完后再恢复被中断进程的现场后，返回到被中断进程。

执行I/O操作

设备驱动程序:与硬件直接相关，用于具体实现系统对设备发出的操作指令，驱动I/O设备工作。

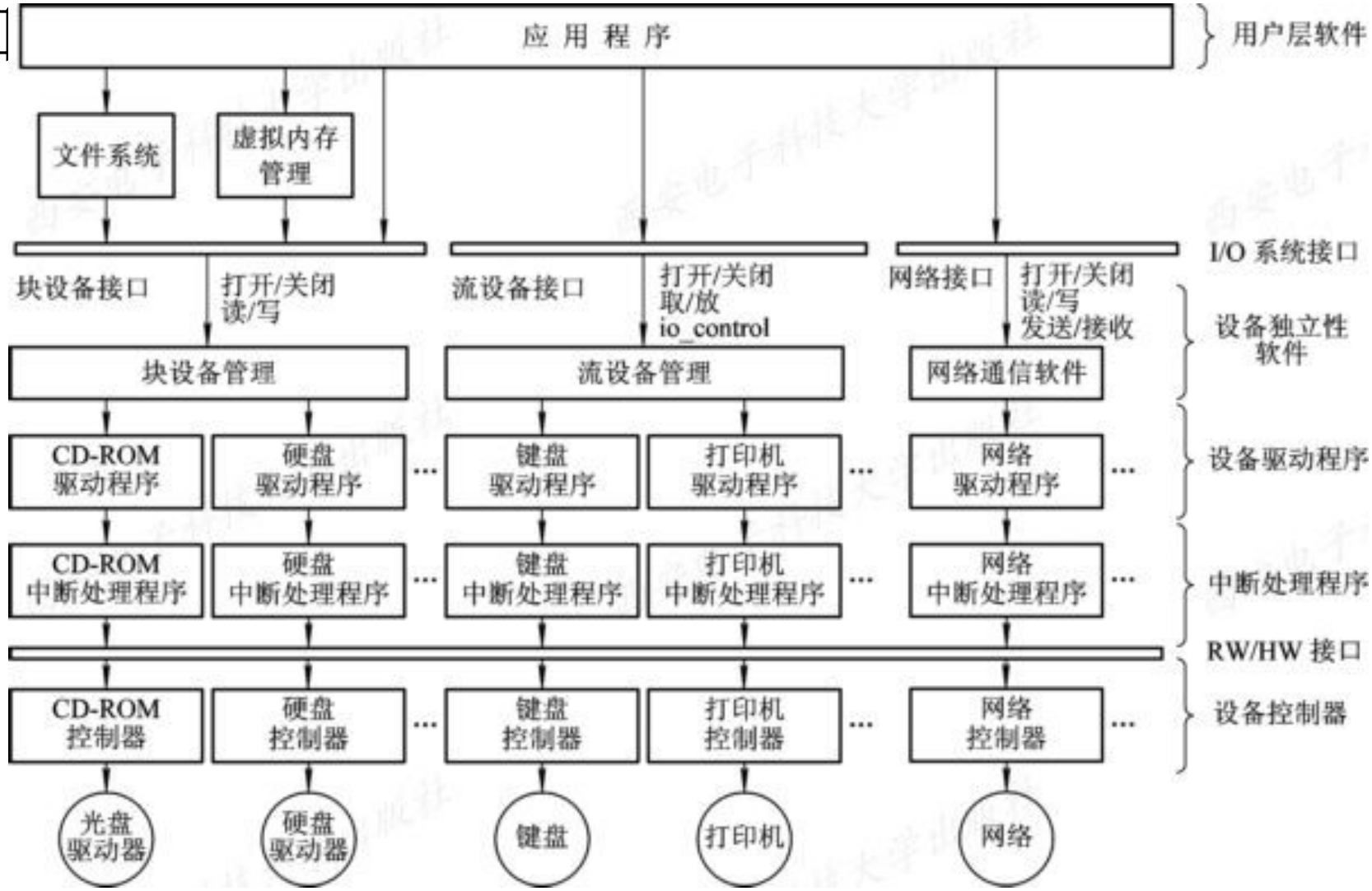
★设备独立性软件:实现用户程序与设备驱动器的统一接口、设备命名、设备的保护以及设备的分配与释放等，同时为设备管理和数据传送提供必要的存储空间。

★用户层I/O软件:实现与用户交互的接口，用户可直接调用在用户层提供的、与I/O操作有关的库函数，对设备进行操作。


应 用 程 序
用户层软件
文件系统
虚拟内存管理
I/O 系统接口
块设备接口
打开/关闭
读/写
流设备接口
打开/关闭
取/放
io_control
网络接口
打开/关闭
读/写
发送/接收
设备独立性软件
块设备管理
流设备管理
网络通信软件
CD-ROM 驱动程序
硬盘 驱动程序
键盘 驱动程序
打印机 驱动程序
网络 驱动程序
设备驱动程序
CD-ROM 中断处理程序
硬盘 中断处理程序
键盘 中断处理程序
打印机 中断处理程序
网络 中断处理程序
中断处理程序
RW/HW 接口
CD-ROM 控制器
硬盘 控制器
键盘 控制器
打印机 控制器
网络 控制器
设备控制器
光盘 驱动器
硬盘 驱动器
键盘
打印机
网络

I/O系统接口

- 块设备接口
- 流设备接口
- 网络接口



2 I/O设备和设备控制器

- 主要内容
- I/O设备
- 设备控制器
- 通道

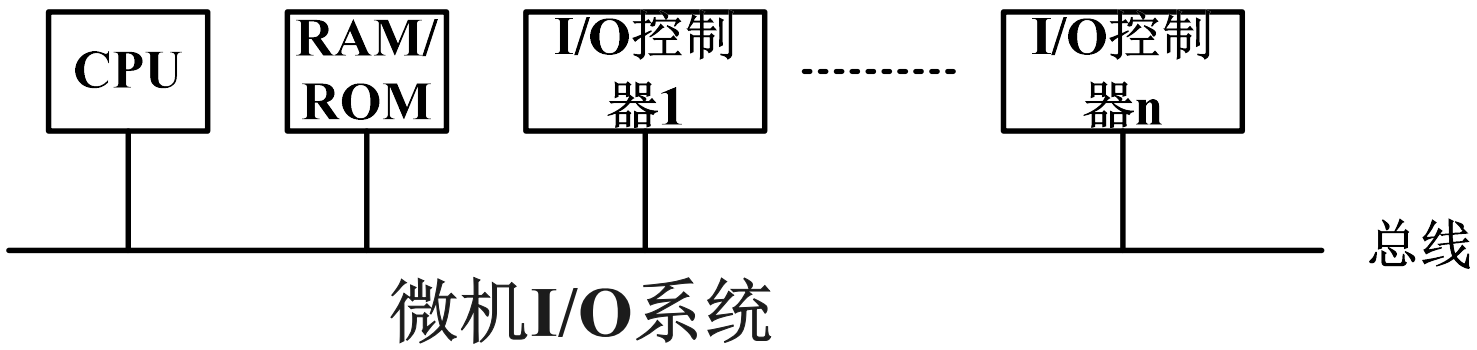
设备

- 设备的分类
  - （1）按速度分：
    - 低：键盘
    - 中：打印机
    - 高：磁盘。
  - （2）按信息交换单位分：
    - 块：磁盘，可定位
    - 字符：打印机、串口

I/O设备

- （3）按设备的共享属性分：
  - 独占：临界资源
  - 共享：磁盘
  - 虚拟：本身固有属性为独占，但将其虚拟为几个逻辑设备。

I/O设备



微机I/O系统

- 微机I/O系统：
- 主机I/O系统：
  - 四级结构：计算机—I/O通道—I/O控制器—设备

- I/O设备
- 设备控制器
- I/O通道
- 总线系统

设备控制器

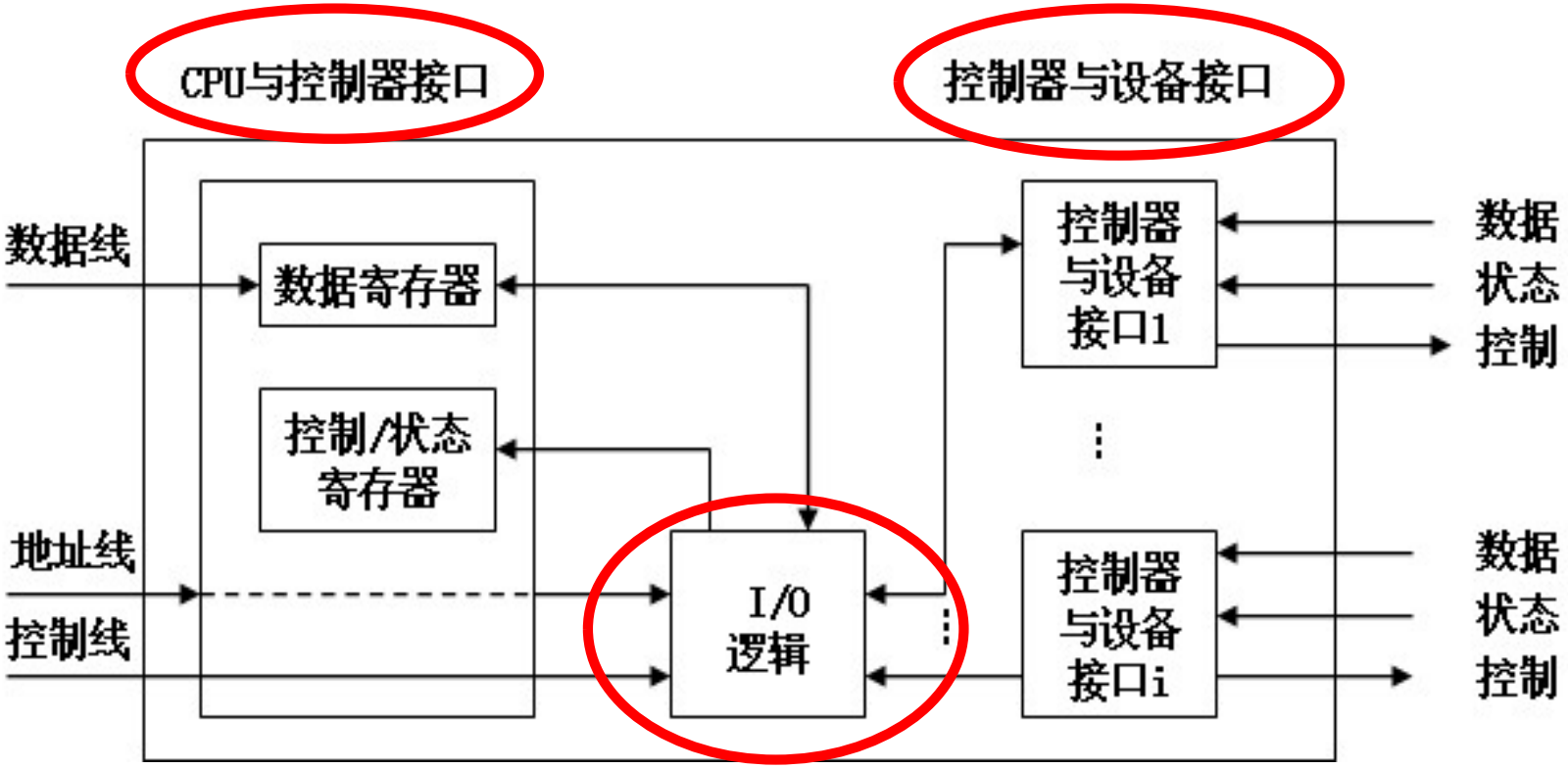
- 可编址设备，控制字符设备和块设备的两类
- 功能：接收CPU命令，控制I/O设备工作，解放CPU.
- 1. 接收和识别命令。
  - 命令寄存器：存放命令
  - 译码器：对接收命令译码
- 2. 数据交换
  - CPU——控制器的数据寄存器——设备
  - 数据寄存器
- 3. 标识和报告设备状态
  - 状态寄存器
- 4. 地址识别
  - CPU通过“地址”与设备通信，设备控制器应能识别它所控制的设备地址以及其各寄存器的地址。
  - 地址译码器
- 5. 数据缓冲
  - 缓冲器
- 6. 差错控制
  - 对数据传送进行差错检测

设备控制器

①与处理机的接口：实现CPU与设备控制器之间的通信，接口中共有三类信号线：数据线、地址线和控制线。数据线通常与数据、控制/状态两类寄存器相连接

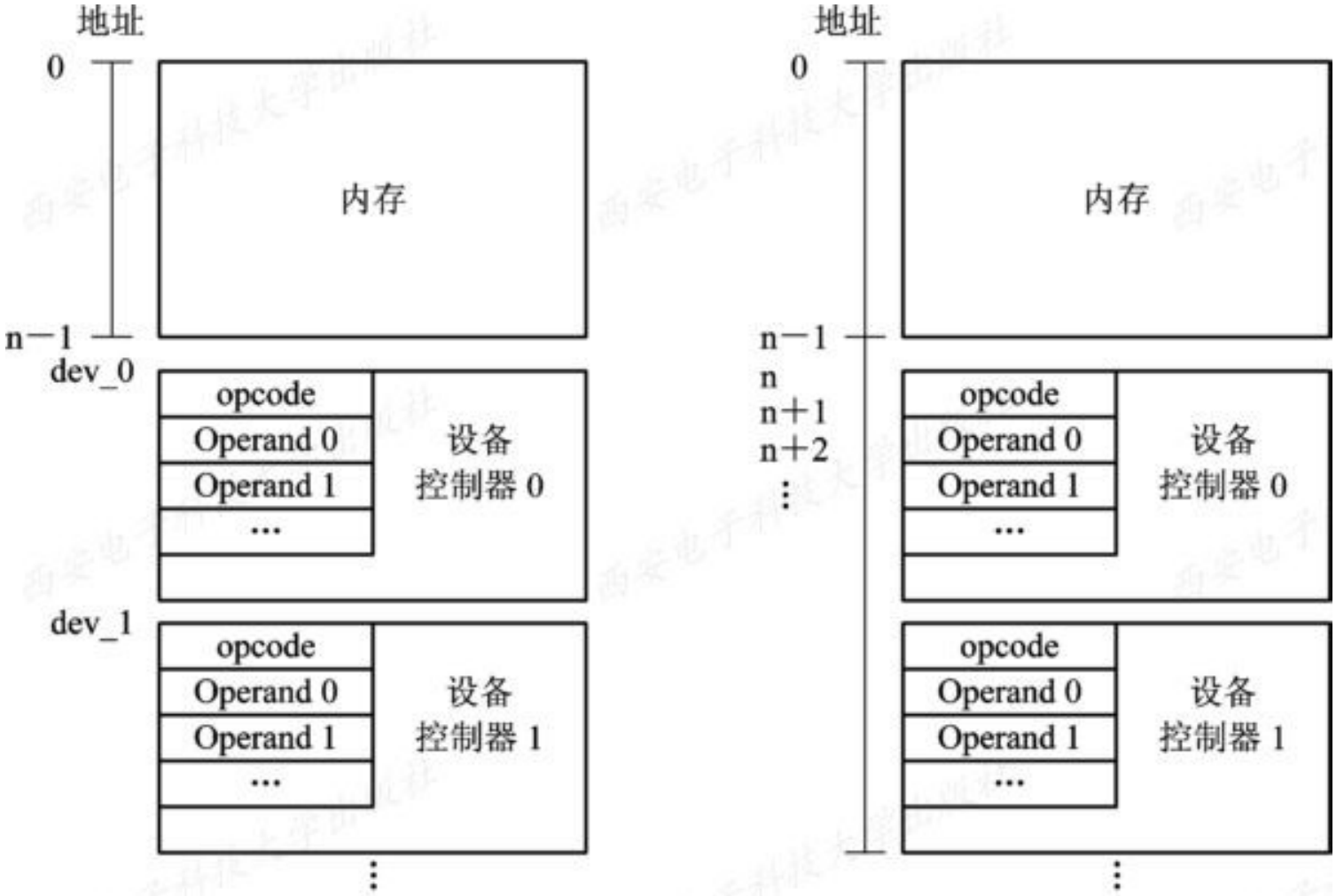
②与设备的接口：控制器中有一个或多个设备接口。在每个接口中都存在数据、控制和状态三种类型的信号。

③I/O逻辑：用于实现对设备的控制，接收CPU命令和地址译码。



内存映像I/O

- 利用特定的I/O指令
- 内存映像I/O



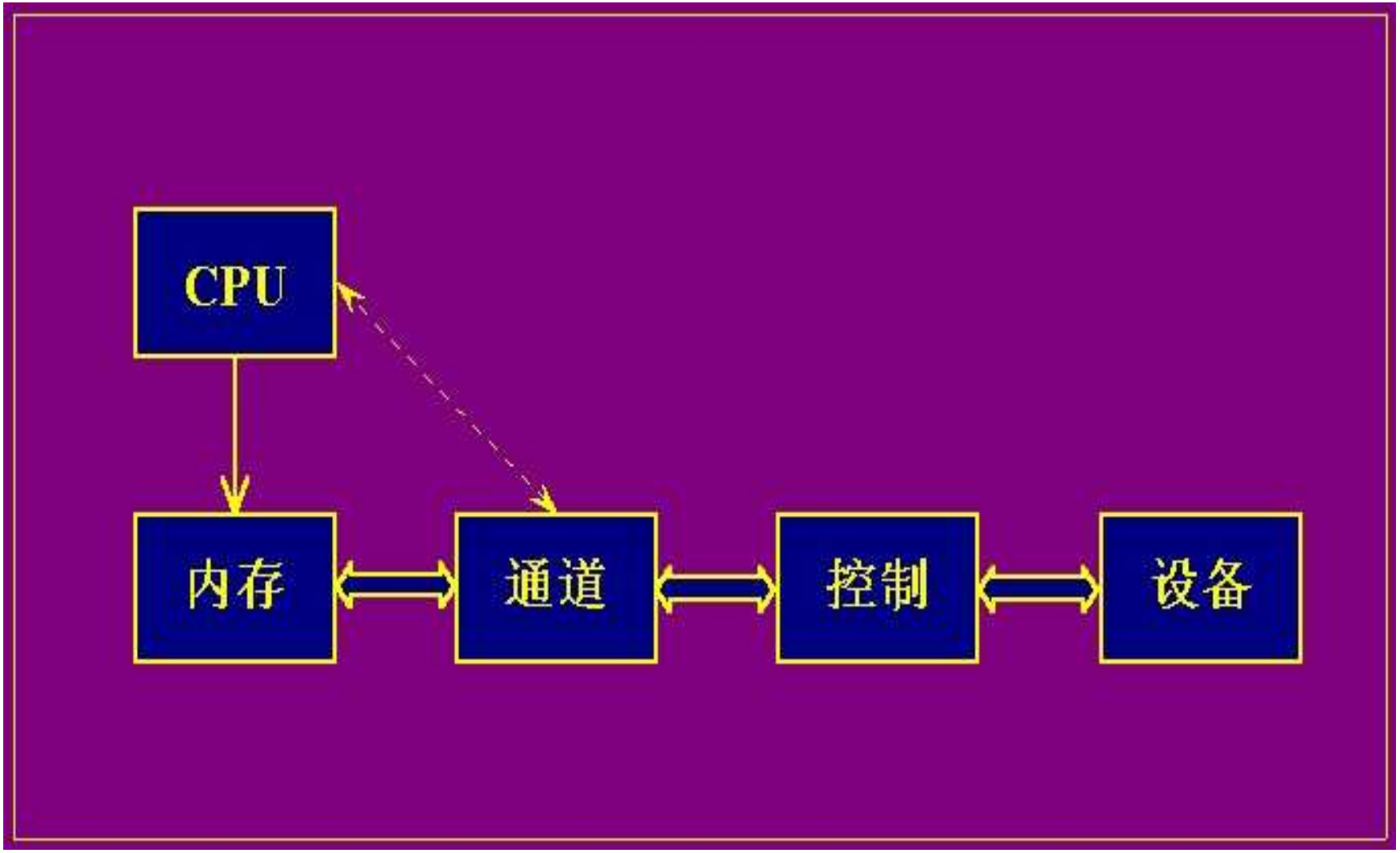
(a) 采用特定的指令形式　　(b) 内存映像I/O形式

I/O通道

- 一、引入
  - 通道

    一种特殊的执行I/O指令的处理机，与CPU共享内存，可以有自己的总线。
  - 引入目的

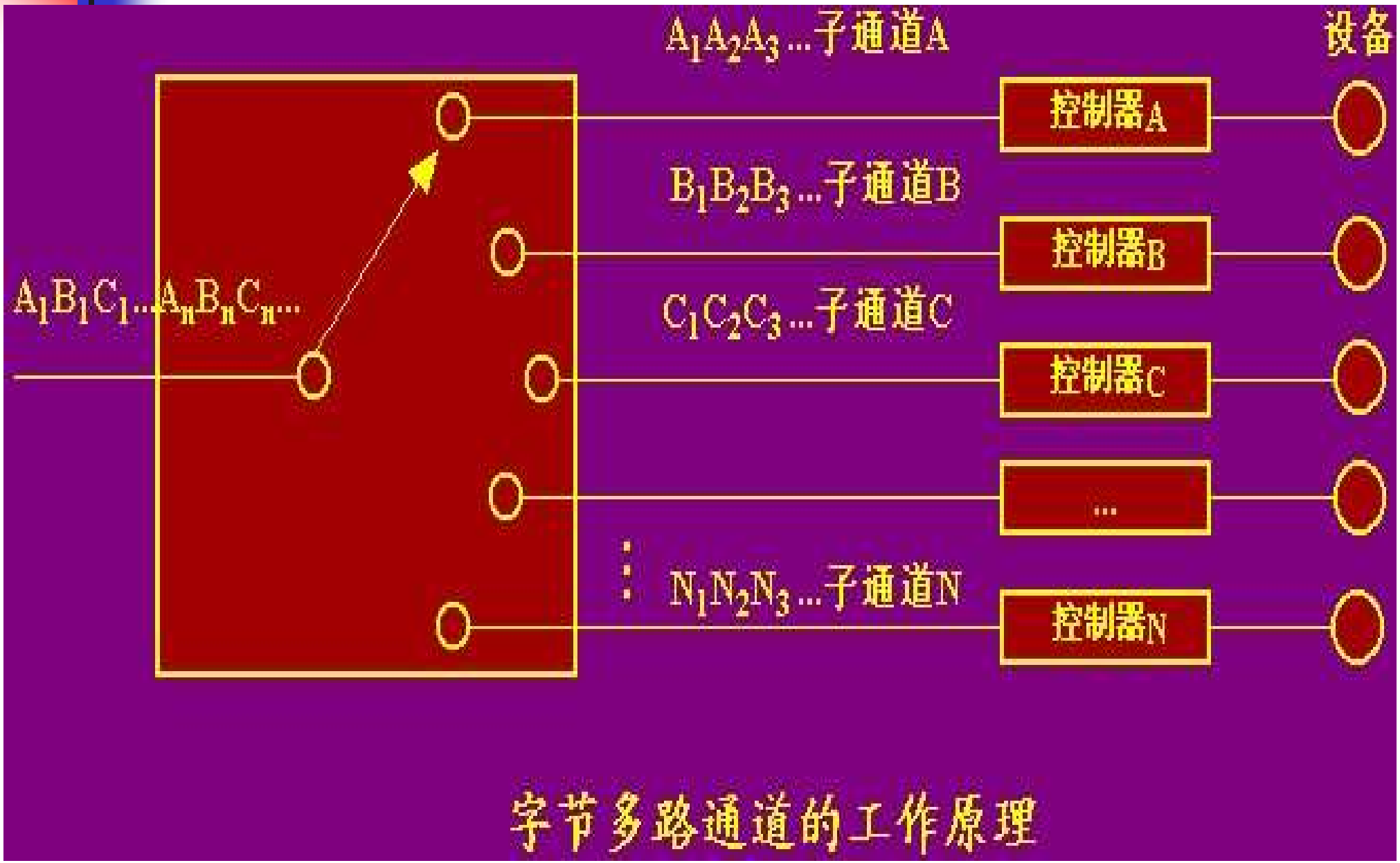
    解脱CPU对I/O的组织、管理。
  - CPU只需发送I/O命令给通道，通道通过调用内存中的相应通道程序完成任务。



硬件连接结构

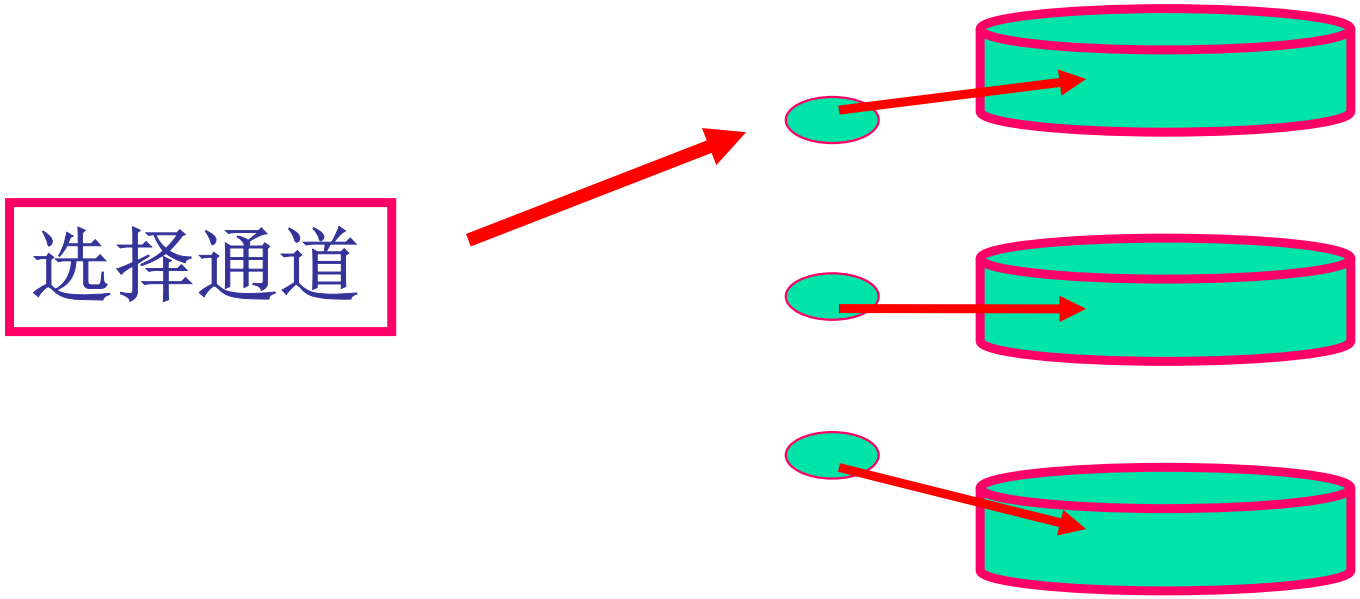
I/O通道

- 二、类型
  - 1. 字节多路通道：
    - 各子通道以时间片轮转方式共享通道，适用于低速设备。
  - 2. 数组选择通道：
    - 无子通道，仅一主通道，某时间由某设备独占，适于高速设备。
    - 但通道未共享，利用率低。
  - 3. 数组多路通道：
    - 多个子通道不是以时间片方式，而是“按需分配”，综合了前面2种通道类型的优点。



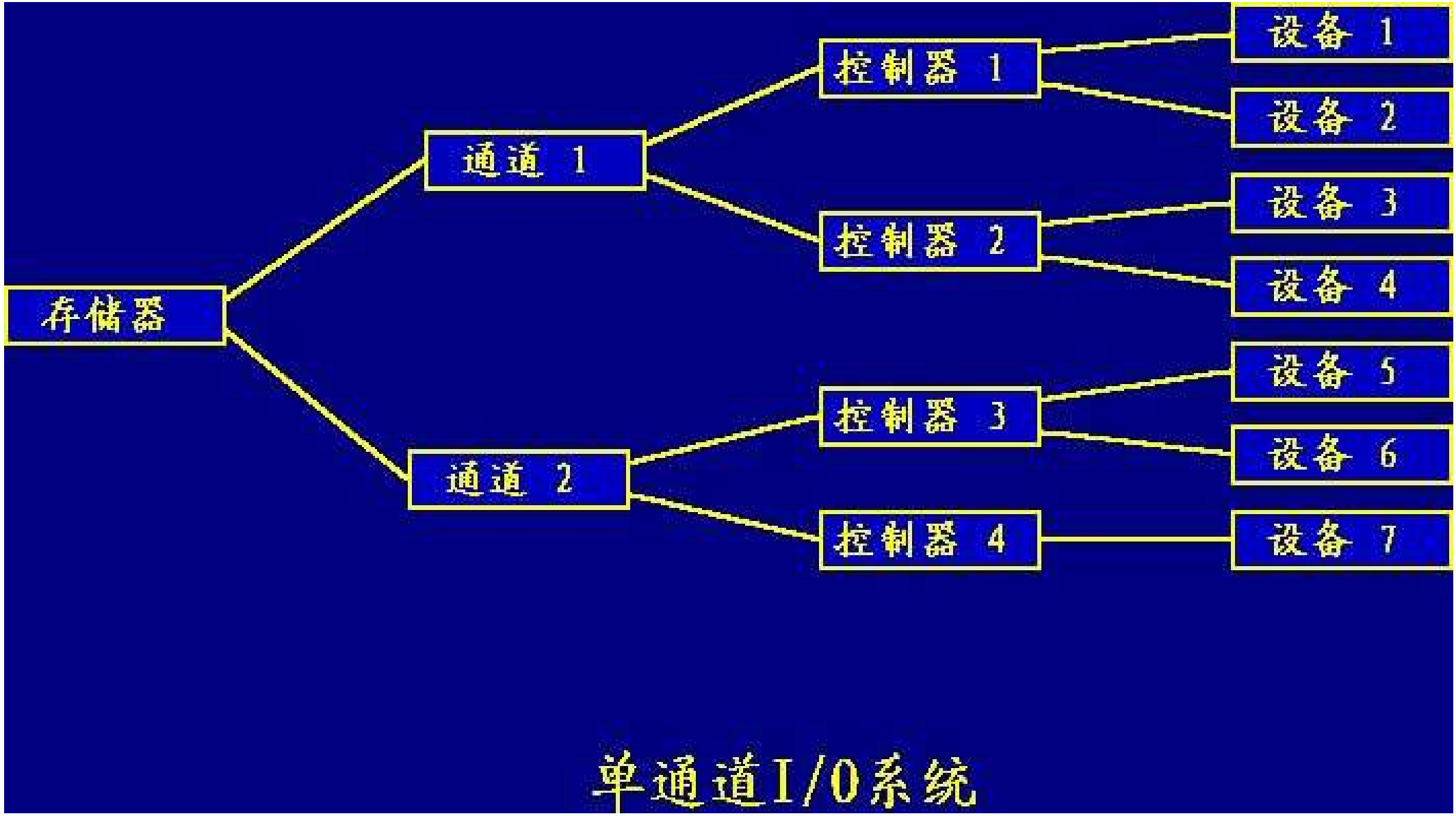
字节多路通道的工作原理

选择通道

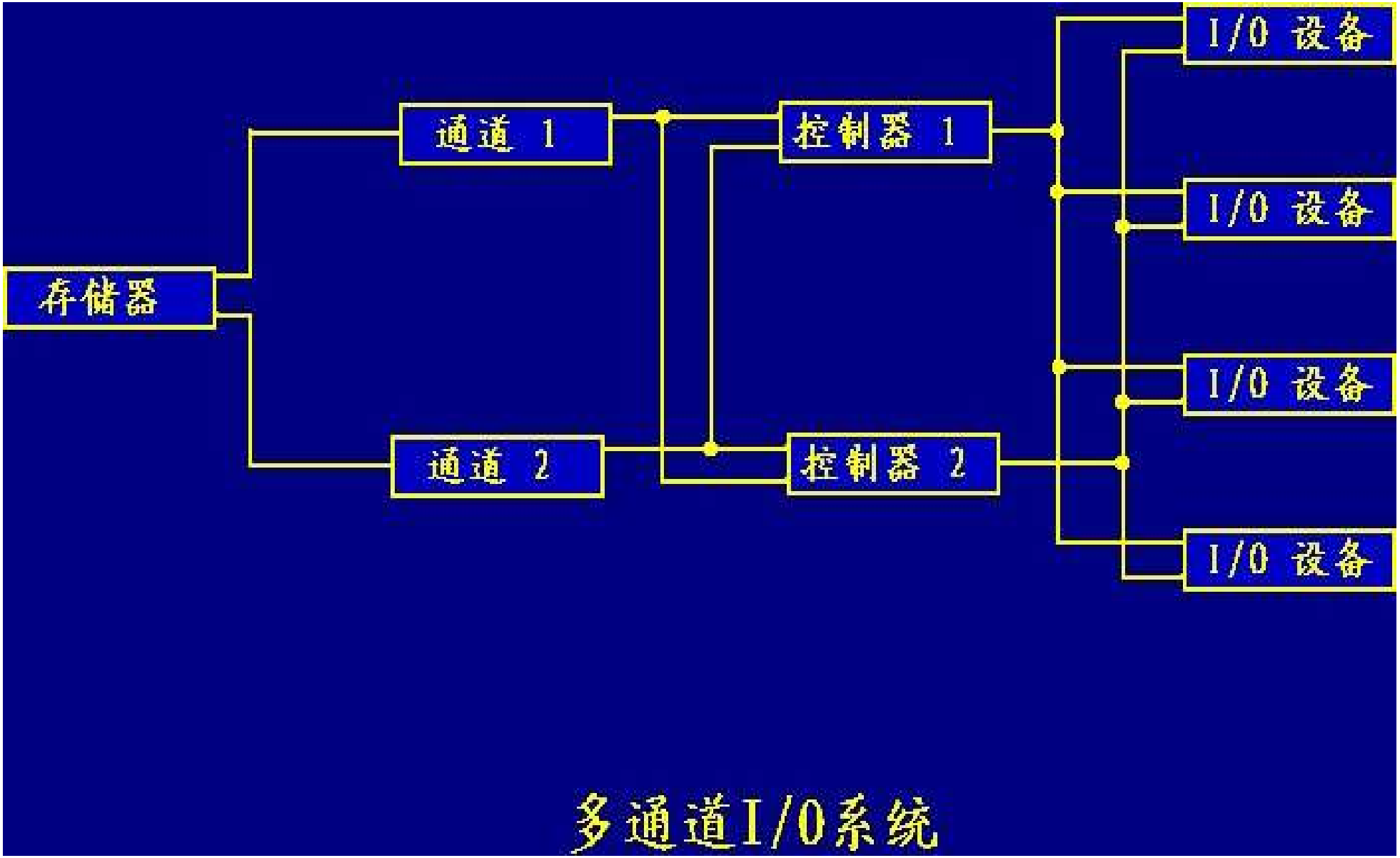


I/O通道

- 三、通道“瓶颈”问题：
  - 解决：采用复联方式
  - 即：增加设备到主机间的通路，而不增加通道。即把一个设备连接到多个控制器上，一个控制器又连接到多个通道上。



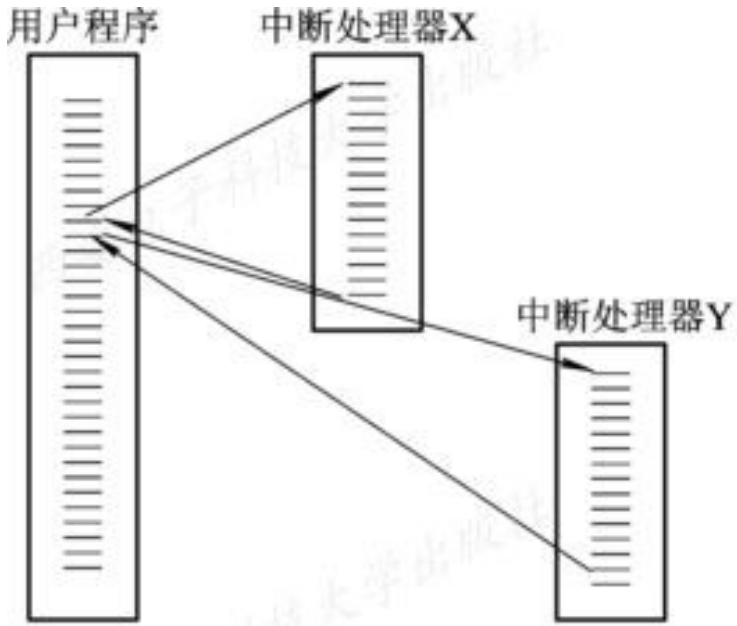
单通道I/O系统



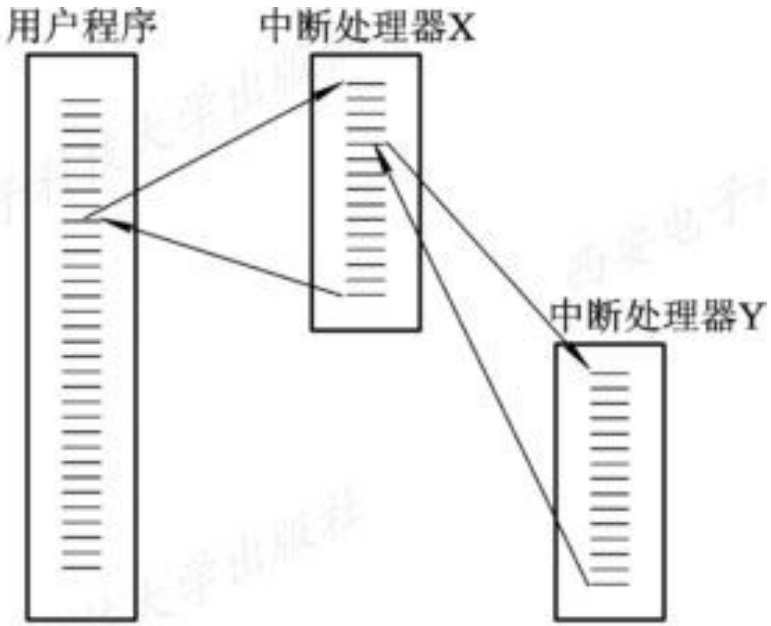
多通道I/O系统

3 中断机构和中断处理程序

- 中断简介
  - 中断（外中断，硬中断），陷入（内中断，软中断）
  - 中断向量表，中断优先级
  - 多中断源处理方式
    - 屏蔽中断
    - 嵌套中断：在设置了中断优先级的系统中



(a) 顺序中断处理



(b) 嵌套中断处理

中断处理程序

- 中断处理程序的处理过程

（1）唤醒被阻塞的驱动程序进程

（2）保护被中断进程的CPU环境

（3）转入相应的设备中断处理程序

  - 由CPU测试各中断源以确定引起本次中断的I/O设备，
  - 并发送一应答信号给发中断请求信号的进程，使之消除该中断请求信号，
  - 然后将该中断处理程序的入口地址装入到程序计数器中，使处理机转向中断处理程序。

（4）进行中断处理

根据设备控制器中的状态字段判断本次中断是：

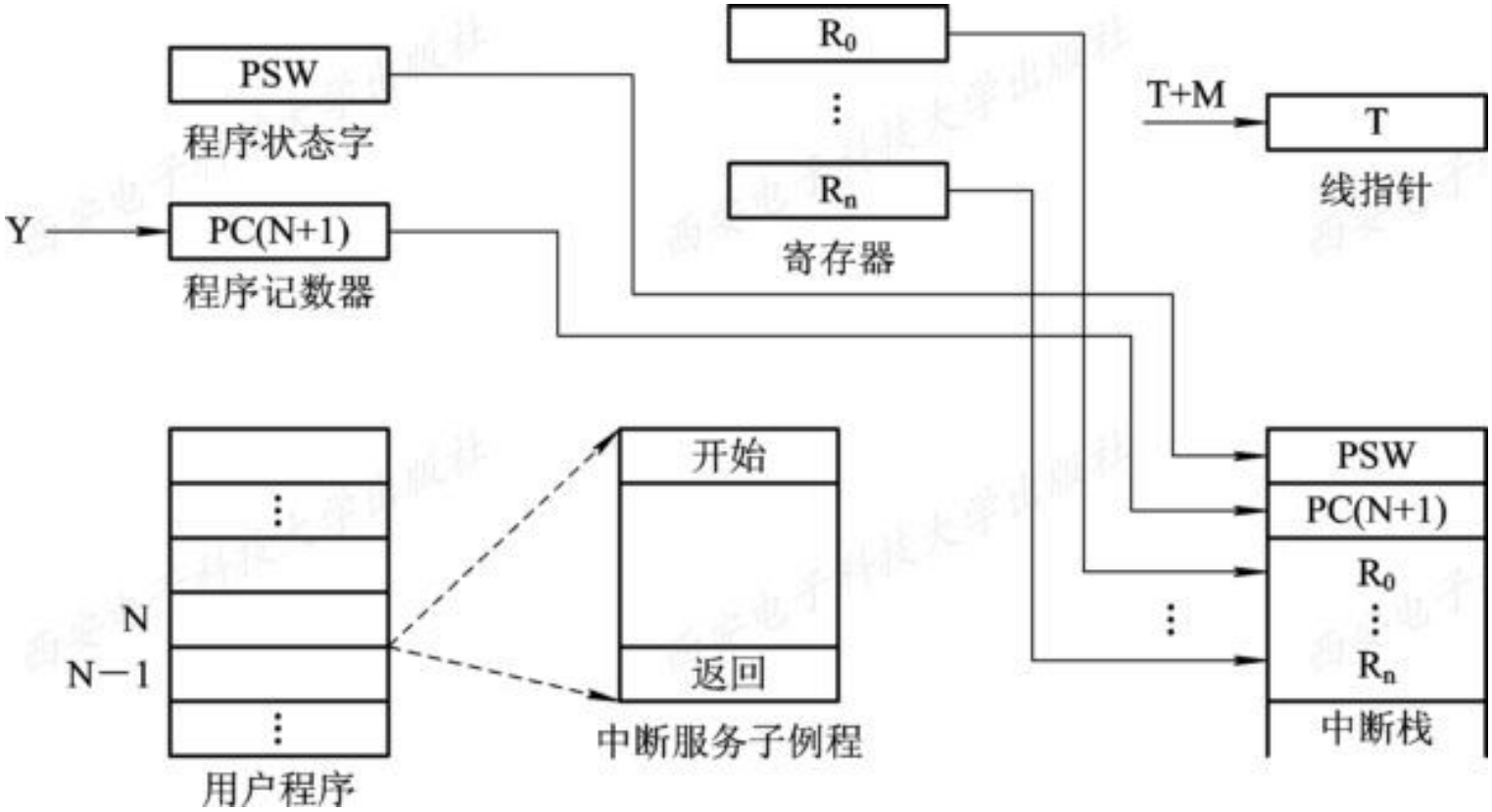
- 正常完成中断？——做结束处理，

  若还有命令，可再向控制器发送新命令进行新一轮的数据传送。

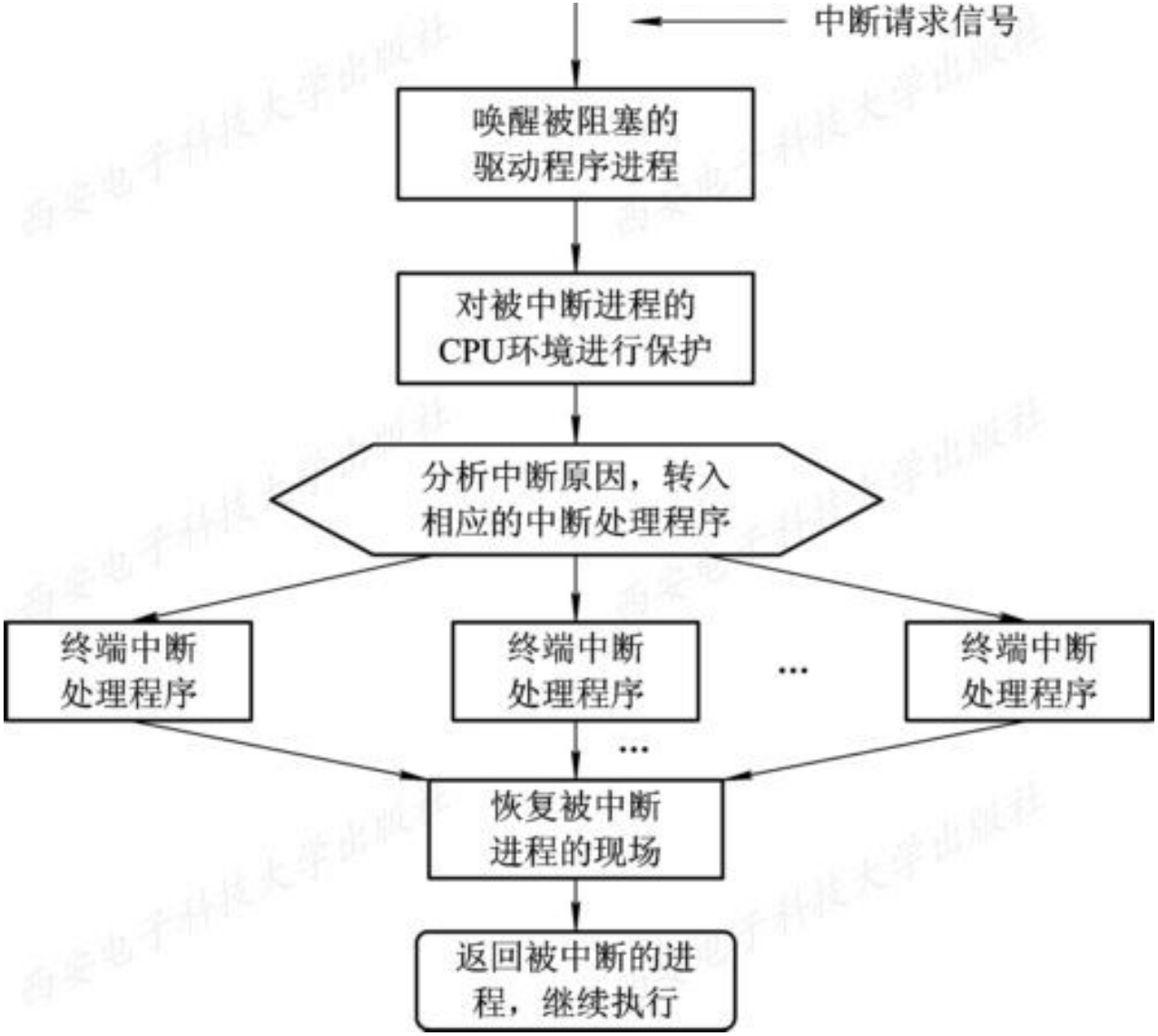
- 异常结束中断？——根据发生异常的原因做处理。

（5）恢复被中断进程的现场

中断现场保护


PSW
程序状态字
Y
PC(N+1)
程序记数器
R_0
R_n
寄存器
T+M
T
线指针
N
N−1
用户程序
开始
返回
中断服务子例程
PSW
PC(N+1)
R_0
R_n
中断栈

中断处理流程


中断请求信号
唤醒被阻塞的
驱动程序进程
对被中断进程的
CPU环境进行保护
分析中断原因，转入
相应的中断处理程序
终端中断
处理程序
终端中断
处理程序
…
终端中断
处理程序
…
恢复被中断
进程的现场
返回被中断的进
程，继续执行

4设备驱动程序

- 设备处理程序即设备驱动程序
  - 是所有与该设备的硬件相关和设备相关的代码的集合，
  - 是I/O进程与设备控制器之间的通信程序。
  - 设备驱动程序在功能是上属于核心，但代码通常由设备制造厂家编写（遵循该操作系统的驱动程序与核心间的接口标准），是在核心之外的一个或多个文件。
- 设备处理的任务：
  - **1**、接收上层软件发来的抽象要求，转换为具体要求后，发送给设备控制器，启动设备去执行；
  - **2**、将设备控制器发来的信号传送给上层软件。
- 驱动程序与硬件密切相关，应为每一类设备或非常类似的两类设备配置一个驱动程序。

设备驱动程序的功能

(1)将接收到的抽象要求转换为具体要求。

(2)检查用户I/O请求的合法性，了解I/O设备状态，传递有关参数，设置设备工作方式。

(3)发出I/O命令，启动分配到的I/O设备，完成指定的I/O操作。

(4)及时响应由控制器或通道发来的中断请求，并根据中断类型调用相应的中断处理程序进行处理。

(5)对有通道的计算机系统，根据用户I/O请求，自动地构成通道程序。

设备驱动程序的特点

(1)它是在请求I/O的进程与设备控制器之间的一个通信程序。

它将进程的I/O请求传送给设备控制器，

而把设备控制器中所记录的设备状态、I/O操作完成情况，反映给请求I/O的进程。

(2)与I/O设备的硬件特性紧密相关。

(3)与I/O控制方式紧密相关。

(4)其中的一部分程序必须用汇编语言书写，另有很多驱动程序的基本部分已经固化在ROM中。

(5)允许可重入

设备处理方式

(1)为每类设备设置一个进程，专门执行该类设备的I/O操作。

(2)在整个系统中设置一个I/O进程，专门负责对系统中所有各类设备的I/O操作。

或设置一个输入进程和一个输出进程，分别处理系统中所有各类设备的输入或输出操作。

(3)不设置专门的设备处理进程，只为各类设备设置相应的设备处理程序(模块)，供用户进程或系统进程调用。

设备驱动程序的处理过程

（1）将抽象要求转换为具体要求

（2）检查I/O请求的合法性

（3）读出和检查设备的状态

（4）传送必要的参数

（5）设置工作方式

（6）启动I/O设备

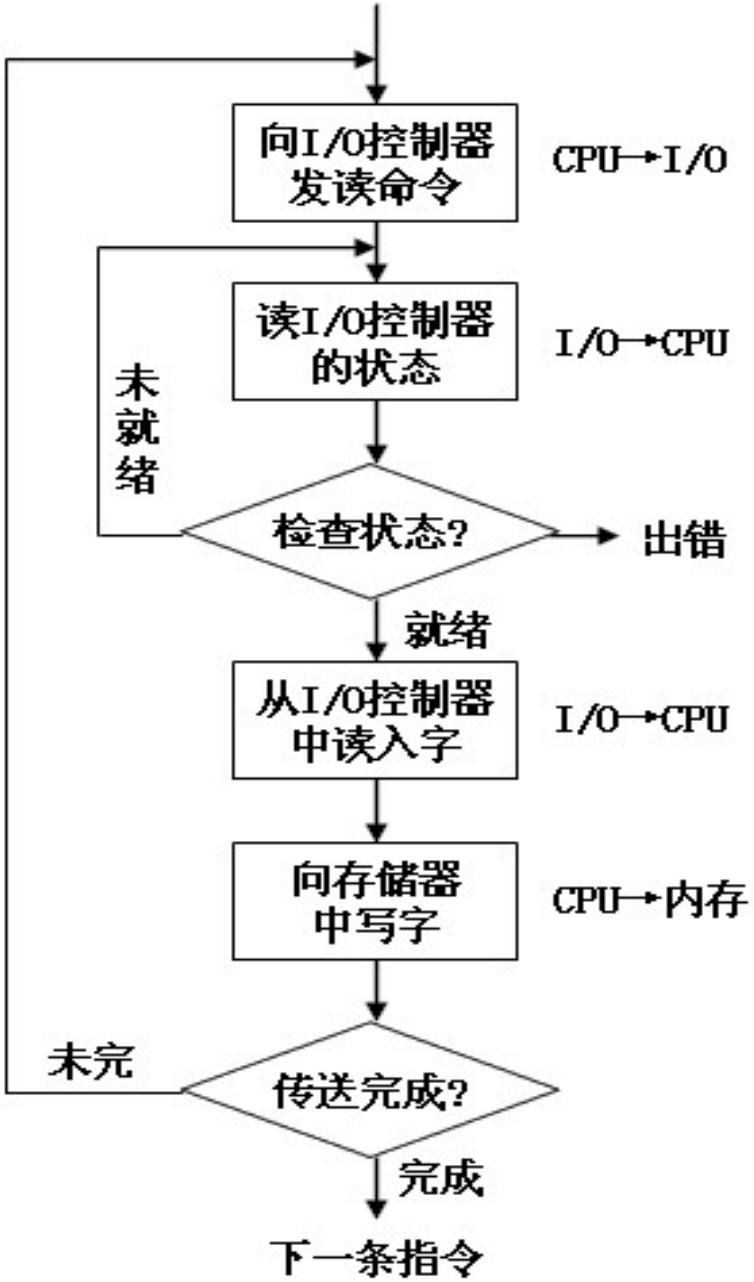
4.3 I/O控制方式

- 四个阶段：
  - 程序I/O——中断I/O——DMA控制——通道控制。
  - 趋势：提高并行度。

1） 轮询的可编程I/O

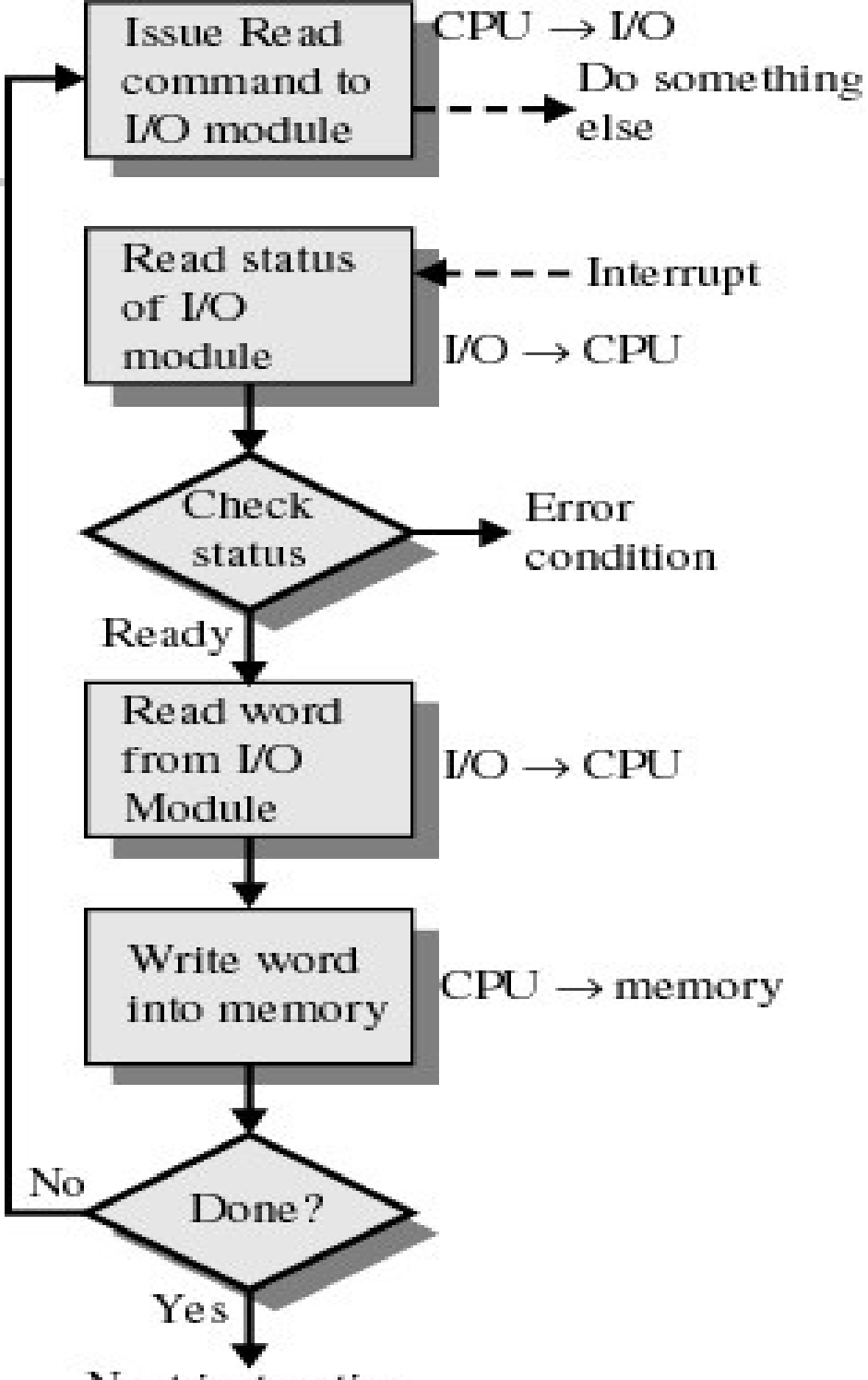
- 程序I/O（忙—等待方式）
- 查询方式：CPU需花代价不断查询I/O状态
- CPU资源浪费极大。
- 例：
  99.9ms+0.1ms=100ms
  99.9在忙等

- 浪费大量**CPU**时间



2） 中断I/O

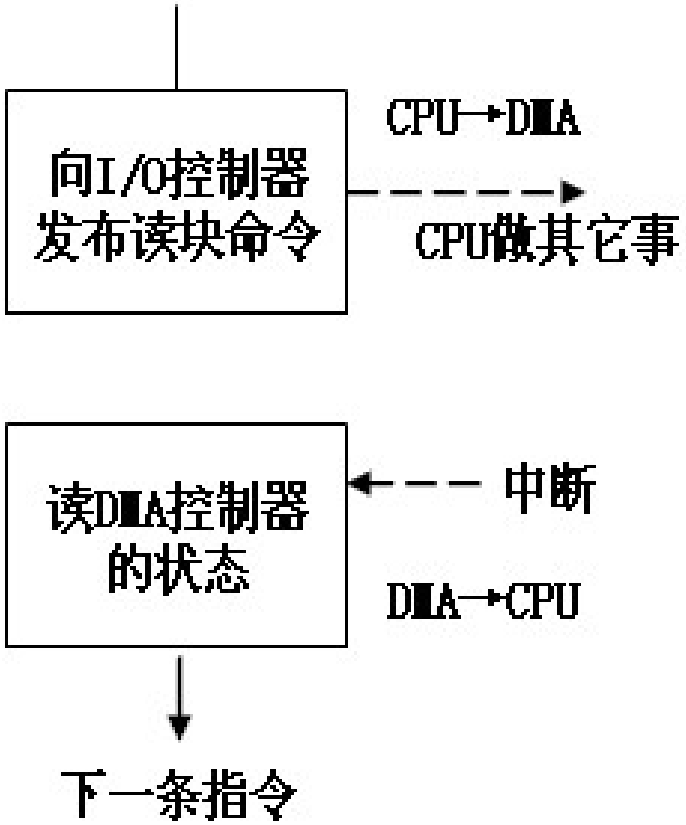
- I/O控制器能发中断
- 向I/O发命令——返回——执行其它任务。
- I/O中断产生——CPU转相应中断处理程序。
- 如：读数据，读完后以中断方式通知CPU，CPU完成数据从I/O——内存
- 缺点：每字节中断，中断次数多



(b) Interrupt-driven I/O

3） 直接存贮器访问方式（DMA）

- 用于块设备中
- DMA：由DMA控制器直接控制总线
- DMA控制器完成从I/O——内存。
- 特点：
  - 数据传输单位：数据块
  - 数据在设备和内存间传送
  - 仅在传送开始、结束时需CPU干预



(c)DMA方式

DMA控制器的组成

①主机与DMA控制器的接口；

②DMA控制器与块设备的接口；

③I/O控制逻辑。

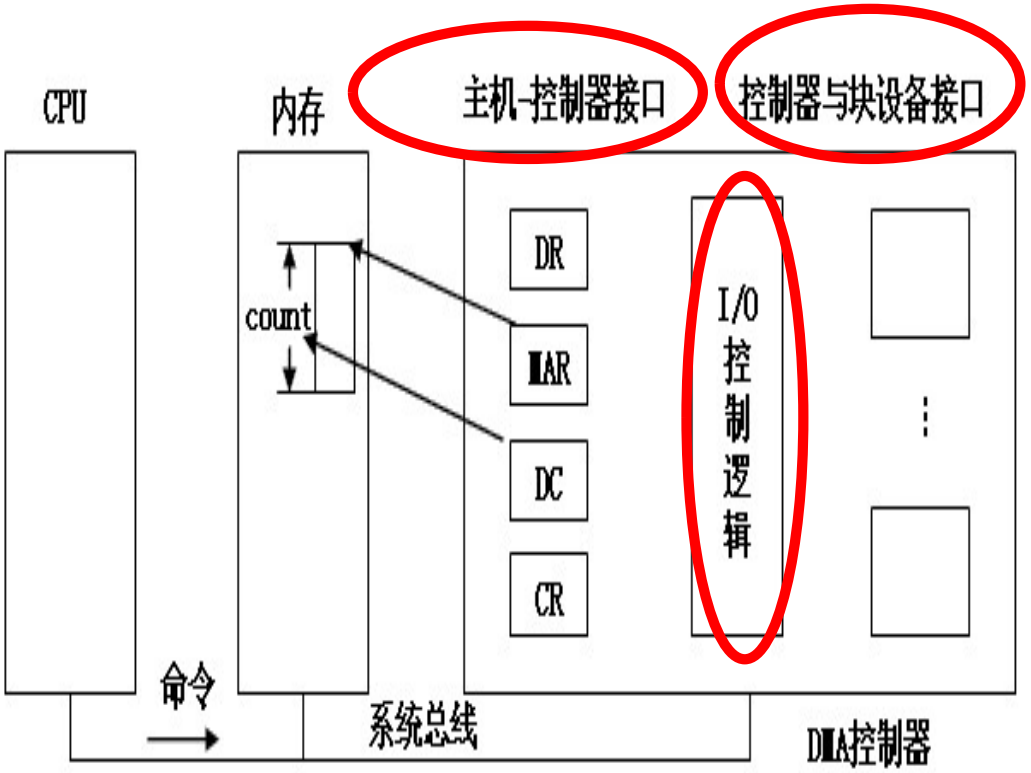
在DMA控制器中有四类寄存器

①命令/状态寄存器CR；

②内存地址寄存器MAR；

③数据寄存器DR；

④数据计数器DC。



工作过程

1、进程（A）要求输入时，把要求传送的内存始址M和要传的字节数送入DMA的内存地址寄存器MAR和传送字数寄存器DC

2、启动设备开始工作。进程（A）挂起。调度另一进程（B）

3、数据输入完成后DMA中断B，转向中断处理程序。

4、中断处理程序唤醒A，返回B的断点继续执行。

5、以后OS调度A运行时，A从M处取数据处理。

DMA方式与中断的主要区别

- 中断方式是在数据缓冲寄存区满后，发中断请求，CPU进行中断处理

  DMA方式则是在要传送的数据全部传送结束时要求CPU进行中断处理

  大大减少了CPU进行中断处理的次数
- 中断方式的数据传送是由CPU控制完成的

  DMA方式是在DMA控制器的控制下不经过CPU控制完成的

4） I/O通道控制方式

- 通道方式：可完成一组块的I/O操作，实现CPU、通道、I/O设备三者并行操作。
- CPU需给出：
  - （1）通道程序首址。
  - （2）要访问I/O设备
- 通道程序：由一系列通道指令组成。
- 通道指令：
  - 操作码，内存地址，计数，通道程序结束位P，记录结束标志R。

5 与设备无关的I/O软件

- 设备独立性
- 概念
  - 设备独立性，又称设备无关性，指应用程序独立于具体使用的物理设备。
- 实现：

（1）引入逻辑设备和物理设备两个概念。

  - 应用程序使用逻辑设备名称请求使用某类设备；
- 系统在实际执行时使用物理设备名称。

（2）系统必须具有将逻辑设备名称转换为某物理设备名称的功能。

两种类型的设备独立性

- 1、独立于同一类设备中的某台具体设备。
  - 系统中有若干台相同的设备，
  - 用户编程时不指定使用哪一个具体的设备，而仅说明要使用哪一类设备，
  - 系统根据当前这类设备的具体状况给用户分配一台具体的设备。
  - 用户不用关心他所使用的到底是哪一台设备。

- 2、独立于不同类型的设备。

例如有一程序要求输入信息，可以从各种不同类型的输入设备上给程序输入数据，则称该程序是独立于不同类型的输入设备的。

又如在MS-DOS系统中，程序的I/O操作不必指出在哪台设备上进行，一般情况下是从键盘上输入数据，而在显示器上输出数据。但用户可以做一次联机操作命令Ctrl+P，则输出数据可以在打印机上打印出来。

实现设备独立性的好处

（1）设备分配灵活

进程以逻辑设备名来请求某类设备时，系统可立即将该类设备中的任一台分配给进程，仅当此类设备已全部分配完毕时，进程才会阻塞。

（2）易于实现I/O重定向

I/O重定向是指用于I/O操作的设备可以更换（即重定向）而不必改变应用程序。

与设备无关的软件

为实现设备独立性，必须在与硬件(或设备)紧密相关的驱动程序之上设置一层软件，称为设备独立性软件。该软件还包括一些共有操作：

（1）向用户层(或文件层)软件提供统一的接口，即各种设备向用户提供统一的接口。例如read、write

以及执行所有设备的公有操作：

（2）缓冲管理。管理字符设备和块设备的缓冲区，提高I／O的效率；

（3)差错控制。处理那些设备驱动程序无法处理的错误。

（4）独占设备的分配与回收；将逻辑设备名映射为物理设备名，并找到相应物理设备的驱动程序；

（5)独立于设备的逻辑数据块

5.4逻辑设备名到物理设备名

（1）逻辑设备表LUT （Logical Unit Table）

- 表目包含：逻辑设备名、物理设备名、设备驱动程序入口地址
- 进程用逻辑设备名来请求分配I/O设备，
- 系统为它分配相应的物理设备，并在LUT上建立一个表目（程序中使用的逻辑设备名、系统分配的物理设备名、设备的驱动程序入口地址）。
- 进程再次利用逻辑设备名请求I/O操作时，系统通过查找LUT即可找到物理设备和驱动程序。

（2）LUT的两种设置方式：

a)整个系统设置一张LUT：

- 不允许有相同的逻辑设备名
- 只适用于单用户系统。

b)为每个用户设置一张LUT：

- 为每个登录用户建立一个进程，同时建立一张LUT放入进程的PCB中。

5.3设备分配

- 为用户或进程分配设备是设备管理的主要功能之一，包括对设备、设备控制器、通道的分配

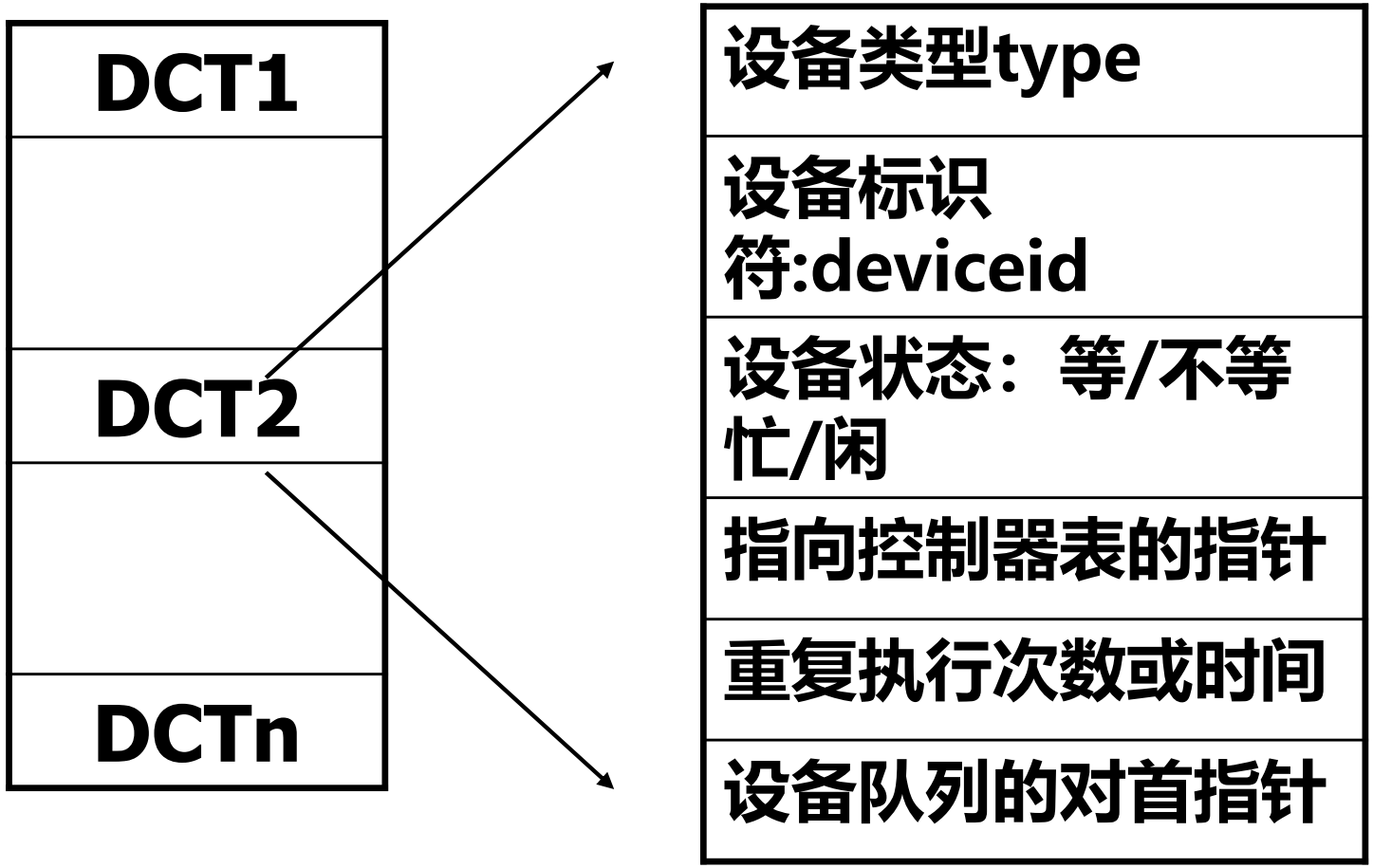
（一）数据结构

- 设备控制表DCT
- 控制器控制表（COCT）
- 通道表（CHCT）
- 系统设备表（SDT）：记录了系统中全部设备及其驱动程序地址。

设备控制表DCT

- 每个设备配置一张设备控制表，记录设备的情况：
  - 设备类型**type**：
  - 设备标识符**deviceid**：
  - 设备状态：处于使用状态时，忙标志置**1**；与设备相连的控制器或通道正忙而不能启动设备，等待标志置**1**。
  - 设备队列队首指针**:**设备请求队列（请求本设备而未得到满足的进程**PCB**按一定策略排成的队列，简称设备队列）的队首指针。
  - 与设备连接的控制器表指针：指向该设备所连接的控制器的控制表。若该设备与多个控制器相连接，应设置多个控制器表指针。
  - 重复执行次数：发生传送错误时重复执行的次数。

设备控制表DCT


DCT1
DCT2
DCTn
设备类型type
设备标识符:deviceid
设备状态：等/不等忙/闲
指向控制器表的指针
重复执行次数或时间
设备队列的对首指针

控制器控制表,通道表和系统设备表

（1）控制器控制表**COCT**

每个控制器设置一张，记录本控制器情况。

（2）通道控制表**CHCT**

每个通道都配有一张通道控制表。

（3）系统设备表**SDT**

- 记录系统中全部设备的情况。
- 每个设备占一个表目（包括设备类型、设备标识符、设备控制表及设备驱动程序入口等信息）。

（二）设备分配时应考虑的因素

- 系统分配设备时应考虑的因素：

(1)设备的固有属性；

(2)设备分配算法；

(3)设备分配的安全性；

（1）考虑设备的固有属性

- 对独占、共享、虚拟三类设备应采取不同的分配策略。

1.独享设备

应采用独享分配策略。但可导致设备利用不充分及引起死锁。

2.共享设备

可同时分配给多个进程，但要合理调度各进程对设备的访问。

3. 虚拟设备

可分配给多个进程，但要控制各进程对设备访问的先后次序。

（2）设备分配算法

- 常用的两种分配算法：

（1）先来先服务

设备队列根据进程对某设备提出请求的先后次序排列

（2）优先级高者优先

设备队列按请求设备进程的优先级由高到低排列，优先级相同的则按先来先服务原则排队。

- 设备分配程序总是把设备首先分配给设备队列的队首进程。

（3）设备分配中的安全性

- 从进程运行的安全性角度看，设备分配有两种方式：
- （1）安全分配方式
  - 进程发出I／O请求后便进入阻塞状态，直到其I／O操作完成时才被唤醒。
  - 所以进程一旦已经获得某种设备(资源)后便阻塞，不会再请求任何资源，而在它运行时又不保持任何资源，因此摒弃了“请求和保持’’条件，分配是安全的。
  - 缺点：进程进展缓慢（CPU与I／O设备串行）。

2．不安全分配方式

进程发出I / O请求后仍继续运行，需要时又可发出其他I / O请求，仅当进程所请求的设备已被另一进程占用时才进入阻塞状态。

- 优点：一个进程可同时操作多个设备使进程推进迅速。
- 缺点：分配不安全。因为有时具备“请求和保持”条件而可能造成死锁。
- 改进：在设备分配程序中增加安全性计算。

（三）独占设备的分配程序

1、基本的设备分配程序

具有通道的系统的设备分配程序分配设备的步骤：

（1）分配设备

（2）分配控制器

（3）分配通道

（1）分配设备

- 根据物理设备名在**SDT**中找出该设备的**DCT**，
- 若设备忙，便将请求进程的**PCB**挂在设备队列上；
- 否则，按照一定的算法来计算本次设备分配的安全性，
  - 若不会导致系统进入不安全状态，便将设备分配给请求进程；
  - 否则，仍将其**PCB**插入设备队列。

（2）分配控制器

- 分配设备给进程后，再到其**DCT**中找出与该设备连接的控制器的**COCT**。
- 若控制器忙，便将请求**I / O**进程的**PCB**挂在该控制器的等待队列上；
- 否则，将该控制器分配给进程。

（3）分配通道

- 分配控制器后，再在**COCT**中找到与该控制器连接的**CHCT**。
- 若通道忙，便将请求**I/O**的进程挂在该通道的等待队列上；
- 否则，将该通道分配给进程。
- 只有在设备、控制器和通道三者都分配成功时，这次的设备分配才算成功；之后便可启动该**I/O**设备进行数据传送。

设备分配程序的改进

- 上述基本的设备分配程序中：
  - (1)进程以物理设备名提出I/O请求；
  - (2)采用的是单通路的I/O系统结构，容易产生“瓶颈”现象。
- 改进
  - （1）增加设备的独立性
  - （2）考虑多通路情况

（1）增加设备的独立性

- 进程用逻辑设备名请求**I/O**。
- 系统先从**SDT**中找出第一个该类设备的**DCT**。如该设备忙，再找第二个，若所有该类设备部忙则把进程挂在该类设备的等待队列上；
- 而只要有一个该类设备可用，系统就可进一步计算分配该设备的安全性。

（2）考虑多通路情况

- 分配控制器和通道时，若设备(控制器)所连接的第一个控制器(通道)忙时，应查看其所连接的第二个控制器(通道)，
- 仅当所有的控制器(通道)都忙时，此次的控制器（通道）分配才算失败，才把进程挂在控制器(通道)的等待队列上；
- 而只要有一个控制器（通道）可用，系统便可将它分配给进程。

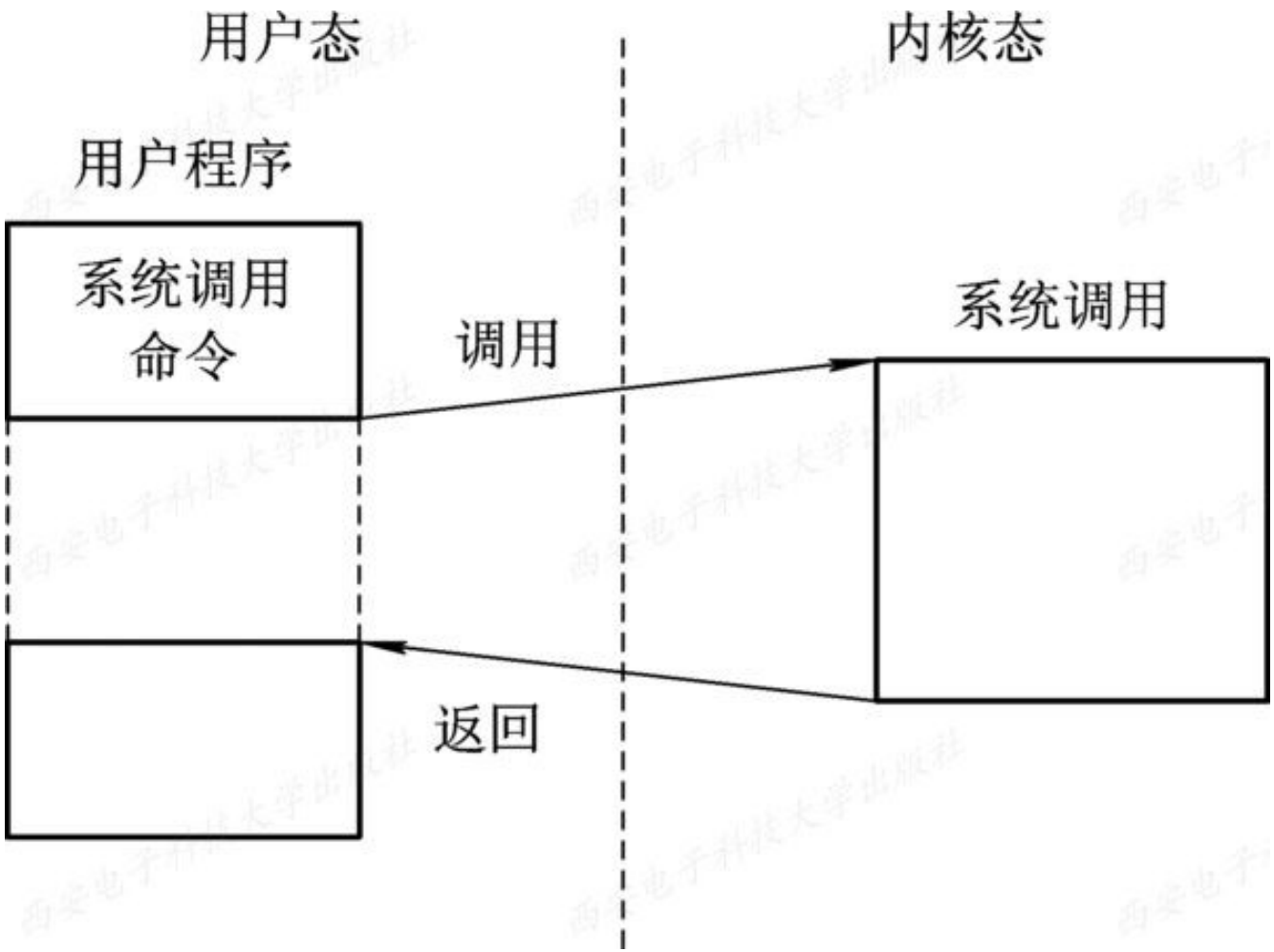
6 用户层的I/O软件

- 系统调用
- **SPOOLing**
- 假脱机打印

6.1 系统调用与库函数

- 应用进程在运行时，必须取得**OS**所提供的服务
- 但安全起见不允许运行在用户态的应用进程去直接调用运行在核心态(系统态)的**OS**过程。
- **OS**在用户层中引入了一个中介过程——系统调用，应用程序可以通过它间接调用**OS**中的**I/O**过程，对**I/O**设备进行操作。

系统调用执行过程



库函数

- 在**C**语言以及**UNIX**系统中，系统调用**(**如**read)**与各系统调用所使用的库函数**(**如**read)**之间几乎是一一对应的。
- 微软定义了一套过程，称为**Win32 API**的应用程序接口**(Application Program Interface)**，该接口与实际的系统调用并不一一对应。
- 用户程序通过调用对应的库函数使用系统调用，这些库函数与调用程序连接在一起，被嵌入在运行时装入内存的二进制程序中。

假脱机技术

- 背景：
  - 在大型、巨型机中，通常采用外围控制机（前端机）来管理I/O。
  - I/O设备不是连接到主机系统，而是连接到外围机上，由外围机来管理。
- 在多道系统中，
  - 利用其中的一道程序模拟脱机输入时的外围控制机的功能，把低速I／O设备上的数据传送到高速磁盘上；
  - 再用另一道程序来模拟脱机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上，
  - 这样，便可在主机的直接控制下，实现脱机输入、输出功能。

SPOOLing

- 概念：
  - 在联机情况下实现的同时外围操作SPOOLing(Simultaneaus Periphernal Operations On-Line)，又称为假脱机操作
- 区别脱机：设备与主机是否连接，目标设备的I/O过程是否在主机控制下进行。
- 注意：SPOOLING只适用可以从程序过程中分离出来的I/O，如打印。
- 作用：通过缓冲方式，将独占设备改造为共享设备。

SPOOLING系统的组成

(1)输入井和输出井

磁盘上的两个大存储空间。

输入井模拟脱机输入时的磁盘，用于收容**I/O**设备输入的数据。

输出井模拟脱机输出时的磁盘，用于收容用户程序的输出数据。

(2)输入缓冲区和输出缓冲区

内存中的两个缓冲区：

输入缓冲区（暂存由输入设备送来的数据，以后再传送到输入井）

输出缓冲区（暂存从输出井送来的数据，以后再传送给输出设备）。

(3)输入进程SPi和输出进程SPo 。

- 进程SPi模拟脱机输入时的外围控制机，将数据从输入设备，通过输入缓冲区再送到输入井。
- 进程SPo模拟脱机输出时的外围控制机，把输出的数据从内存送到输出井，再待输出设备空闲时，将输出井中的数据，经过输出缓冲区送到输出设备上
- CPU直接从输入井读数据入内存，数据直接输出到输出井。

（4)井管理程序

例子

- (1)输入
  - a. 进程n请求－－> SPi为n在输入井中分配空间——>设备数据由输入buf送输入井——>生成输入请求表挂输入请求队列。
  - b. CPU空——取请求表中的任务,送进程缓冲区。
- （2）输出：（打印）
  - a. 进程n请求——>SPo为n在输出井中分配空间——>将数据由进程空间转到输出井——>生成一打印请求表挂打印请求队列。
  - b. 打印机空——>查打印请求表中的任务——> 取输出井中的数据——>到输出buf ——>打印

SPOOLING的特点

（1）提高了I/O速度

对低速设备操作——>变为对输入/出井中数据的连续存取。

（2）将独占设备改造为共享设备

分配设备的实质是分配输入/出井

（3）实现了虚拟设备功能

每个进程独占了一个（逻辑上的）设备，实现了将独占设备变换为若干台对应的逻辑设备的功能。

注意：并没有提高**CPU**利用率，而是增加了**CPU**的负担，但提高了系统吞吐量和所有资源的利用率。

假脱机打印机系统

- 假脱机打印系统主要有以下三部分：
  (1) 磁盘缓冲区。
  (2) 打印缓冲区。
  (3) 假脱机管理进程和假脱机打印进程。
- 当用户进程请求打印时，假脱机管理进程在输出井中为之申请一空闲盘块区，将要打印的数据送入其中；再为用户进程申请一张空白的用户请求打印表，将用户的打印要求填入其中，再将该表挂到请求打印队列上。
- 如果还有进程要求打印输出，系统仍可接受该请求。
- 真正的打印输出由假脱机打印进程完成。

守护进程(daemon)

为打印机建立一个守护进程，由它执行一部分原来由假脱机管理进程实现的功能，如为用户在磁盘缓冲区中申请一个空闲盘块，并将要打印的数据送入其中，将该盘块的首址返回给请求进程。

另一部分由请求进程自己完成，每个要求打印的进程首先生成一份要求打印的文件，其中包含对打印的要求和指向装有打印输出数据盘块的指针等信息，然后将用户请求打印文件放入假脱机文件队列(目录)中。

守护进程是唯一允许使用打印机的进程。

7 缓冲区管理

- 缓冲的引入：
  - 1. 缓和CPU和I/O设备间速度不匹配的矛盾。

    如：计算——打印buffer——打印
  - 2. 减少对CPU的中断频率

    如：buffer越大，“buffer满”信号发生频率越低。
  - 3. 解决数据粒度不匹配问题
  - 4. 提高CPU和I/O并行性
- 缓冲的主要目的是为了提高中央处理机与外设的并行程度。

常用的缓冲技术

1、单缓冲

2、双缓冲

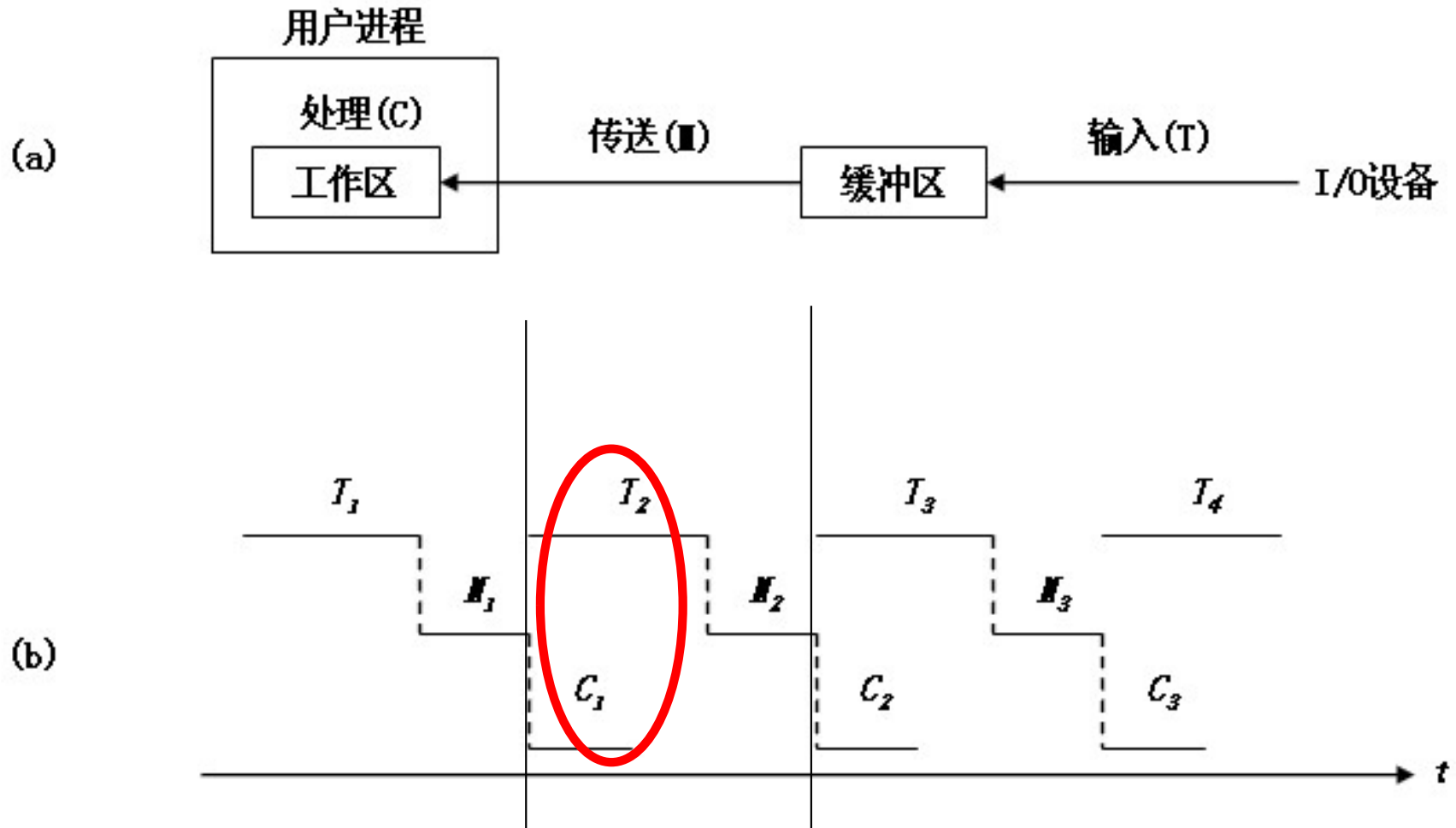
3、循环缓冲

4、缓冲池

1）单缓冲

当进程发出一**I/O**请求时，**OS**为之分配一缓冲区。（使用单缓冲时必须互斥）

系统对每一块数据的处理时间为**Max(C，T)+M**

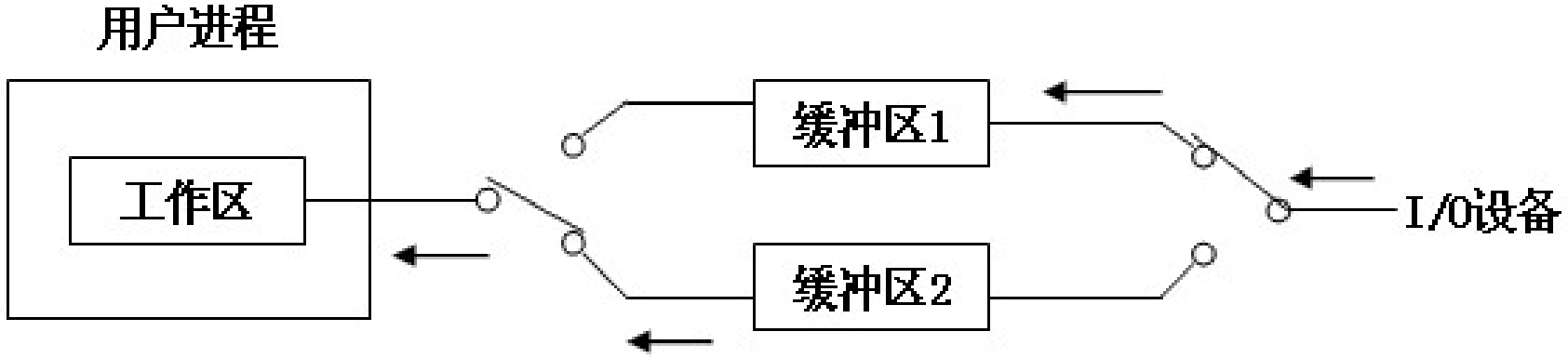


2）双缓冲技术

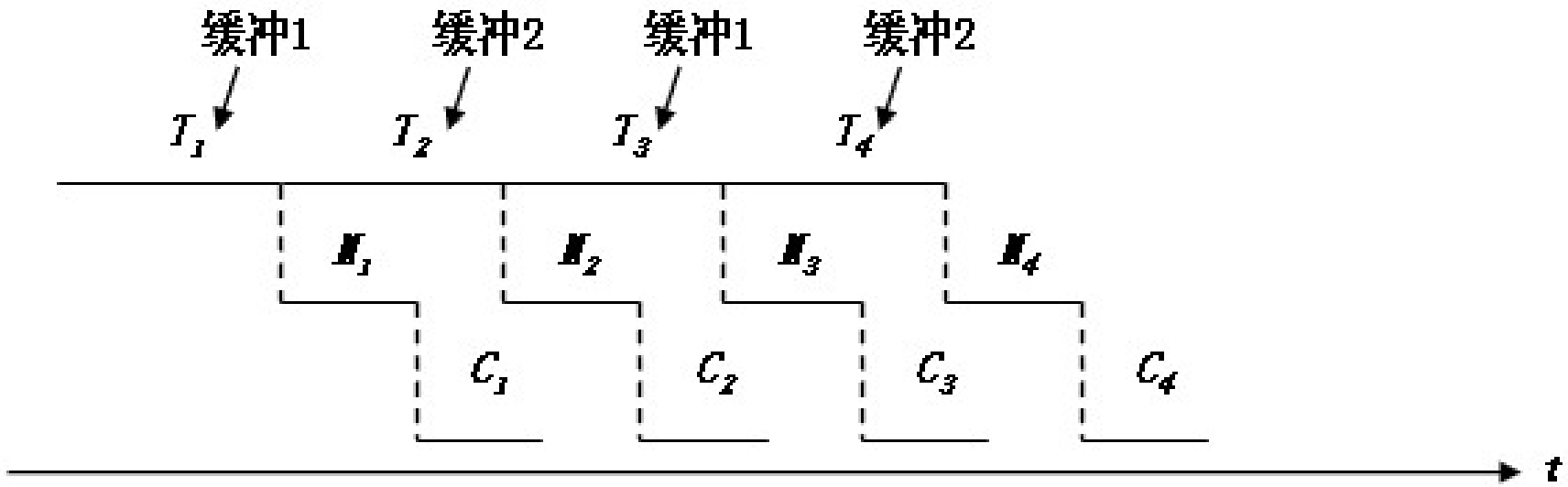
为了加快输入输出速度，交替使用两个缓冲区，使**CPU**和设备的并行操作的程度进一步提高。

系统处理一块数据的时间可粗略地认为是**Max(C，T)**

(a)



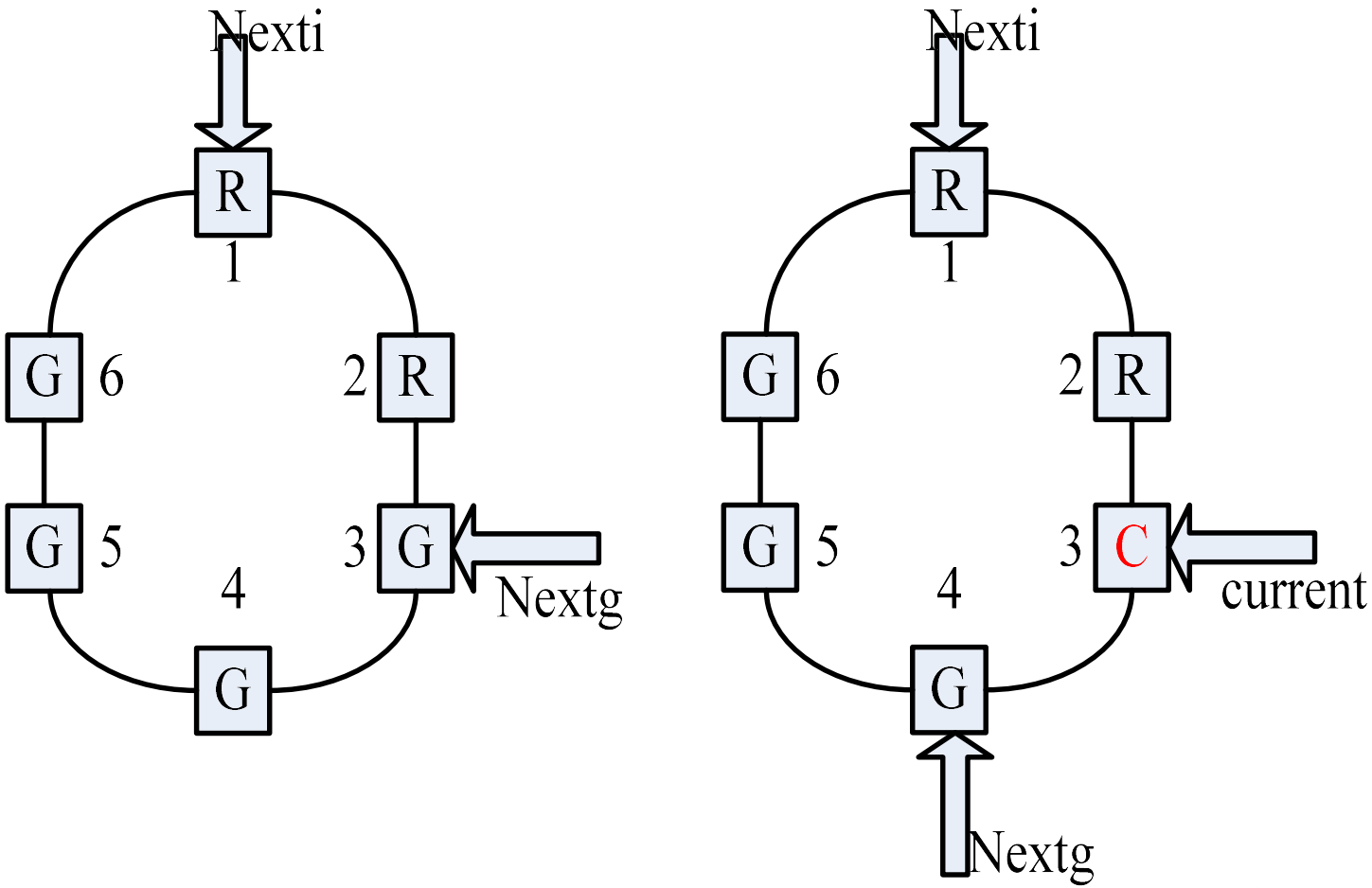
(b)



3） 环形缓冲（循环缓冲）

- 分析双缓冲：
  - 当生产和消费数据的速度基本匹配时能获得较好效果。
  - 但若两者速度相差甚远时，效果不太理想。
  - 但随着缓冲区的数量增加，使情况有所改善。
- 因此引入循环缓冲。
  - 三种类型缓冲区
  - 三种指针

- 循环多缓冲图



三种类型缓冲区：R:空缓冲；G：满缓冲；C：当前缓冲

三种指针：

循环多缓冲的使用

- $next_g$：指示下一个应取数据的buf
- $next_i$：指示下一个空buf.
- Getbuf：
  - 取$next_g$对应缓冲区提供使用，将$Next_g$置为空，$Next_g$=（$Next_g$+1）Mod N
  - 将$Next_i$对应缓冲区提供使用，将$Next_i$置为满，$Next_i$=（$Next_i$+1）Mod N
- Releasebuf:
  - 若C满，则改为G；
  - 若C空，则改为R；

循环多缓冲的同步问题

- Nexti 追上Nextg：

  表示输入速度>输出速度，全部buf满，这时输入进程阻塞。

  系统受计算限制

- Nextg追上Nexti：

  输入速度<输出速度，全部buf空，这时输出进程阻塞。

  系统受**I/O**限制

4）缓冲池

- 分析环形缓冲区：
  - 一般用于特定的进程，属于专用缓冲区，
  - 当系统较大时，会有许多环形缓冲区，要消耗大量的内存空间，利用率不高。
- 为提高缓冲区的利用率，使用公用缓冲池，池中的缓冲区可供多个进程共享。
- 缓冲池由内存中一组大小相等的缓冲区组成，池中各缓冲区的大小与用于I/O的设备的基本信息单位相似
- 缓冲池中各缓冲区可用于输出、输入信息，并可根据需要组成各种缓冲区队列。
- 缓冲池属于系统资源，由系统进行管理。

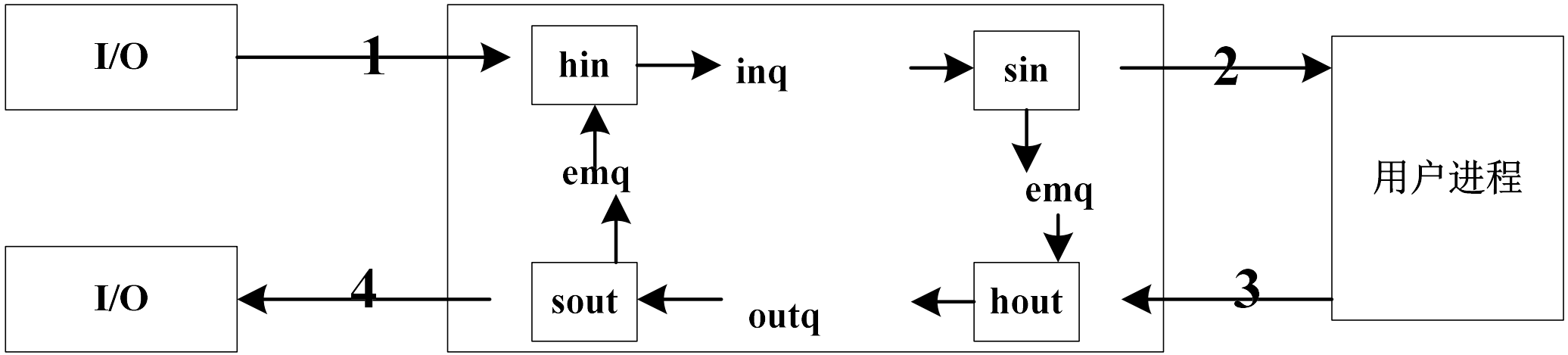
缓冲池(Buffer Pool)的组成

- 输入、输出的公用缓冲池，应含有三种类型的缓冲区（1）空闲缓冲区(2)装满输入数据的缓冲区；(3)装满输出数据的缓冲区。
- 三个队列：

  （1）空缓冲队列emq。由空缓冲区链成的队列。

  （2）输入数据队列inq。由装满输入数据的缓冲区链成的队列。

  （3）输出数据队列outg。由装满输出数据的缓冲区所链成的队列。
- 四种工作缓冲区：收容输入数据的；收容输出数据的；提取输入数据的；提取输出数据的。



四种工作方式

1.收容输入；hin=getbuf(emq); putbuf(inq,hin)

2.提取输入 sin=getbuf(inq);计算； putbuf(emq,sin)

3.收容输出；hout=getbuf(emq); putbuf(outq, hout)

4.提取输出 sout=getbuf(outq);输出；putbuf(emq,sout)

Getbuf和Putbuf过程

```
Getbuf(type)
Begin
  wait(RS(type));
  wait(MS(type));
  B(number):=takebuf(type);
  signal(MS(type));
end
```

```
Putbuf(type)
Begin
  wait(MS(type));
  addbuf(type,number);
  signal(MS(type));
  signal(RS(type));
end
```

互斥：为每个缓冲队列设置一个互斥信号量MS(type)。
同步：为每个缓冲队列设置一个资源信号量RS（type）